# Swingup control and stabilization of a Furuta pendulum

Louis Callens, Mathias Bos, Alvaro Florez, Mathias Schietecat,
Joris Gillis, Wilm Decré

## 1 Introduction

The aim of this assignment is to swingup a Furuta pendulum and stabilize it at the inverted position using an MPC controller. The Furuta pendulum is described using four states in total, representing angles of the motor and the pendulum ($\theta_1$ and respectively $\theta_2$) and the angular velocities (represented by $\dot{\theta}_1$ and $\dot{\theta}_1$). More precisely, the goal is to drive the state to a reference state given by

$$x = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \quad u = \ddot{\theta}_1 \quad \longrightarrow \quad x_{\text{ref}} = \begin{bmatrix} 0 \\ \pi \\ 0 \\ 0 \end{bmatrix} \quad u_{\text{ref}} = 0,$$

or equivalently, to drive an error defined by

$$\delta x = x - x_{\text{ref}} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} - \begin{bmatrix} 0 \\ \pi \\ 0 \\ 0 \end{bmatrix}$$

to zero. However, since $\theta_i' \coloneqq \theta_i + 2k\pi$ (for $k \in \mathbb{Z}$) is equivalent to $\theta_i$, the error can be defined as

$$\delta x \coloneqq \begin{bmatrix} \cos(\theta_1) \\ \cos(\theta_2) \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} - \begin{bmatrix} \cos(0) \\ \cos(\pi) \\ 0 \\ 0 \end{bmatrix}.$$

The dynamics of the Furuta pendulum are derived in [1] (see equation (31)). In this assignment, we assume acceleration control of the motor meaning we neglect any torques and assume $\ddot{\theta}_1 = u$. This leads to dynamics given by

$$f(x,u) = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ u \\ -p_1\cos(\theta_2)u + \frac{1}{2}\left(\dot{\theta}_1\right)^2 \sin(2\theta_2) - p_2\dot{\theta}_2 - p_3\sin(\theta_2) \end{bmatrix}$$

with some parameters defined as

$$p_1 = \frac{m_2 l_1 l_2}{\hat{J}_2}\left[\frac{kg \cdot m^2}{kg \cdot m^2}\right] \quad p_2 = \frac{b_2}{\hat{J}_2}\left[\frac{N \cdot m \cdot s/rad}{kg \cdot m^2}\right] \quad p_3 = \frac{g m_2 l_2}{\hat{J}_2}\left[\frac{m^2 \cdot kg/s^2}{kg \cdot m^2}\right].$$

Using the experimental setup, numerical values as shown in Table 1 have been identified.

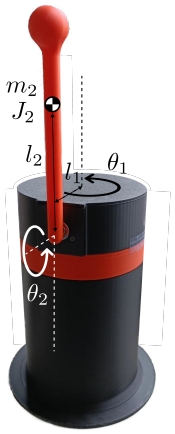


Figure 1: Furuta pendulum

| $p_1$ | $p_2$ | $p_3$ |
|---|---|---|
| 0.54895841 | 0.35365408 | 65.69229089 |

Table 1: Identified parameter values for the experimental setup

## 2 Assignment

In this assignment, we will define an MPC-controller in `python` using Impact. This will be done in the file `py/furuta_pendulum_swingup_impact.py`. After implementing an MPC-controller, the optimization problem is solved and a visualization is generated in `py/figs/furuta_swingup_open_loop.png`.

Additionally, a closed-loop simulation can be performed (if `CL_SIM = True` at the top of the file). This will solve the MPC optimization problem in every simulation step and apply it to the system. A 3D visualization is generated in a local browser window (if `ENABLE_3D_VISUALIZATION = True` at the top of the file). In the terminal, a link to the window will be pasted. This should be open/refreshed before pressing enter to start the closed-loop simulation. A summary visualization is generated in `py/figs/furuta_swingup_closed_loop.png`.

1. A simple optimization problem can be defined as in (1), where an initial state $x_0$ is given and deviation of the reference is penalized.

$$\begin{aligned} \underset{x(t),u(t)}{\text{minimize}} \quad & \int_{t_0}^{t_0+T} \left( ||\delta x||_Q^2 + ||u||_R^2 \right) dt && \text{(1a)} \\ \text{subject to} \quad & x(t_0) = x_0, && \text{(1b)} \\ & \dot{x} = f(x,u), && \text{(1c)} \\ & u_{\min} \leq u \leq u_{\max} && \text{(1d)} \end{aligned}$$

   Note that $||v||_A^2 \coloneqq v^\top A v$ and $Q$ and $R$ are tunable matrices that are typically diagonal. Find suitable weight matrices $Q$ and $R$. Does the controller behave as expected? Why (not)?

2. Add a terminal constraint stating

$$x(t_0 + T) = \begin{bmatrix} 0 & \pi & 0 & 0 \end{bmatrix}^\top.$$

What is the effect of this constraint? Does this constraint improve the performance?

3. To incentivize the controller to perform a swingup motion, an additional cost can be introduced to track a desired reference energy level. Define

$$E(x) = \frac{1}{2} p_1 \left(\dot{\theta}_2\right)^2 + p_3(1 - \cos(\theta_2))$$

as the energy in the system for a given state vector $x$. The reference energy is then given by

$$E_{\text{ref}} = 2p_3. \tag{2}$$

This leads to the definition of optimization problem (3) given by

$$\begin{aligned}
\underset{x(t),\,u(t)}{\text{minimize}} \quad & \int_{t_0}^{t_0+T} \left( ||\delta x||_Q^2 + ||u||_R^2 + \left(E(x) - E_{\text{ref}}\right)^2 \right) dt && \text{(3a)} \\
\text{subject to} \quad & x(t_0) = x_0, && \text{(3b)} \\
& \dot{x} = f(x, u), && \text{(3c)} \\
& u_{\min} \leq u \leq u_{\max} && \text{(3d)}
\end{aligned}$$

Implement this optimization problem.

4. Add noise to the simulation by adding `simulator.set_noise_std(0.005)`. Additionally, introduce a plant-model mismatch by defining

```
p1_sim = p1
p2_sim = p2
p3_sim = p3 * 1.05
```

Is your controller robust to noise and modeling errors? What goes wrong? Can you make your controller more robust?

5. Replace the earlier definition of $E_{\text{ref}}$ in (2) by

$$E_{\text{ref}} \coloneqq 2p_3 \cdot 1.1. \tag{4}$$

Does this make the controller more robust? Can you explain why?

6. Export your controller using impact to obtain the C-artifact. You can do this by stating `EXPORT = True` at the top of the file.
After a successful export, you should see a new directory called `furuta_pendulum_build_dir` containing all exported artefacts.

7. Now we can use the c-artifact to perform a real-time simulation in `C++`. The file `src/sim_swingup_mpc.cpp` defines a simulation model using `CasADi`, creates a simulator object, an estimator and a controller. These `C++` objects are defined in `ImpaC++t` and allow you to easily a simulator thread and a controller thread in real-time. To run it, first we need to build the executable using `CMake` and we need to tell `ImpaC++t` where to find the artefact by adding it to our path.
   In a terminal, execute `./run_cpp_simulation.sh`. This script will perform these steps and run the simulation. It will also run python scripts to visualize the result.

8. If your controller is not behaving as intended, an example solution is provided as well for you to compare. It is located on the solution branch of the git repository called `furuta_pendulum_swingup_impact_mpc_solution.py`.

9. With a working simulation script using `ImpaC++t`, the step to deployment is small. Consider the file `src/deploy_swingup_mpc.cpp`. Instead of a simulator, it defines an object of type `FurutaStepperIO` which will communicate with the hardware. Additionally, instead of applying acceleration $\ddot{\theta}_1$, the motor is velocity-controller. Therefore, we integrate the control signal outputted by MPC controller and compute a desired velocity $\dot{\theta}_1$.

# References


[1] Benjamin Seth Cazzolato and Zebb Prime. "On the Dynamics of the Furuta Pendulum". In: *Journal of Control Science and Engineering* 2011.1 (2011), p. 528341. DOI: https://doi.org/10.1155/2011/528341. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1155/2011/528341. URL: https://onlinelibrary.wiley.com/doi/abs/10.1155/2011/528341.